类型安全 10.29

lec02 scala 基础 10.29

scala 支持各种各样的风格，当然我们下面只会介绍函数式编程。。

声明式编程：是我们整个类型安全系统最核心的概念。就是，这是一切的基础，因为我们下面居然不会用到例如 for 循环之类的过程式。我们之所以要求声明式编程，正是因为我们采用的函数式编程中**函数作为一等公民**可以很好的让声明式在 map、flatmap、for-yield 中借助函数实现几乎所有功能（而避开传统的过程式的编程）。这个概念确实会在下面重复提到。

val 和 var：val 是一个不可变的值 value，var 是一个可变的变量 variable。我们下面会要求，不能使用任何变量，也即，我们希望采用一种“事件溯源”的核心思想，保留中间的每一个步骤，这样可以在 code review 时更容易看懂意图（这个是不是真的不知道），以及在遇到问题时能够更容易定位和修复（这个是真的）。

但是这里就容易有一个问题，如果我全部都用的是 val，我总会有需要跨项目通用的信息，我们会在下面说函数式编程的**纯函数**中说到这件事情，这种需要调用外界信息的读写的操作通常涉及数据库或网络调用，我们统称是**脏操作**，后面的许多设计都是为了让这种脏操作在边界才统一产生。

当然这里一个直接的解释办法是，例如银行存款，我们在数据库中存储记录，例如建户、存钱取钱，查询余额时将记录加和。当一定存储量时做快照 snapshot（类似于定期重构，但是仍然保留所有历史记录）。这里的例子其实比较浅层，我们的终极目的是类型安全。

val 的基础类型：Int、Long、Double、Float、字符串。

在**声明式编程**中我们不使用 for 循环，我们使用 map/flatMap/filter 代替 for 循环（看起来很难代替？但是这也是我们一定要搭配**函数式编程**的原因）。我们下面称一个 collection（数据包袱）是一个打包了一些数据的东西，其实就对应 python 中的 list、dict、array、set 等，总之就是把一堆同类型的数据打包成了一个整体。

`.map` 其实相当于遍历 list 中的每个元素然后做一个 map 中的映射处理。例如 `val a = List(1,2,3).map(i => i*2)`。函数式编程将函数视作一等公民，所以我们可以通过 `.map()` 里面放一个函数（这里其实等价于一个返回两倍自身值的函数）来实现 for 的一些功能。

List 具有一些基本操作（就像 python 中的 list 容器一样），一个 val 的 List 例如 `val a = List(List(1,2),List(3,4))` 有 `a.flatten` 会把这个 List 展开成 `List(1,2,3,4)`。也即，`.flatMap` 相当于 `map + flatten`。

最后我们再次声明，在函数式编程中，函数是一等公民，我们上面其实也在试图表达把行为（函数）像数据一样传递的便利性，这看起来很灵活。（当然看起来 python 和 c++ 都可以做到，c++ 可能稍微复杂一点，但是 scala 的独特之处就在于，首先**函数就是对象**因为 scala 底层的函数其实是 FunctionN 特质的实例，然后**类型严谨性** scala 有非常强大的类型推断系统这是 python 绝对不具备的，以及**语法糖极简**我们下面会看到的）

---

lec03 scala 基础 + 函数式编程 10.29

模式匹配：使用 `match + case x => ...` 即可。当然 Scala 中有各种各样的模式匹配。

捕获异常 `try, catch, finally` 其实和 python 是一样的，但是我们下面会有更好的工具所以我们不用这个传统的东西。

`Option[T]` 其实是一个**封装好了的捕获异常的盒子**，里面有 `Some(value)` 和 `None`（即一个包含单个元素或者空的快递箱，快递箱这个名词用的很贴切）。我们用这个东西的原因是，如果我们报出异常例如 null，在外面的每一层我都需要用例如 if 来判定是否可以继续操作，而 `Option[T]` 可以简化这个过程，如果这个盒子已经是空的就 continue 操作，如果这个盒子在执行过程中出了问题就 None 然后 continue 操作，我们无需每次判断是否能操作，我们就把这个盒子当作一个正常的变量操作就行。这个 `Option[T]` 看起来有点像**面向对象编程 OOP** 的设计，但是是合理的，把“抛异常”破坏了控制流的糟糕设计 $\rightarrow$ “返回一个特殊对象”符合 OOP 的优雅设计。

`Try[T]` 和 `Option[T]` 其实是一个东西，只不过 `Try[T]` 里面有 `Success(i)` 和 `Failure(s)` 包含输入或者失败信息（如果失败会允许贴一个失败信息的封条）。

`object`（我们这里说的是语法上的 object 不是后面说的对象）本身是**整个项目中的一个单例**，即全局只有一个（建议是，我们可以将一些工具定义在 object 里面）。当然 scala 程序的 main 入口其实也定义在 object 里面。

class 是类，和 python 中的 class 一样，类是一种模板，我们可以批量生成 class。但是事实上我们在这里不提倡使用 class（因为可变），我们提倡下面的 case class。

case class 是**样例类**，即**专门承载数据的不可变类**。简单来说生成了 case class 一个对象之后里面的值就是初始值不能做任何改动了。

枚举类型 enum 定义有限状态，enum 可以带上参数，即枚举类型可以**携带数据**。

这里介绍两个不在一个层级但是有点相近的概念，一个是 **Abstract class** 即抽象类，不需要实例化，我们通常会让 case class 继承某个抽象类，类似于“狗”继承“动物”一样。还有一个是 **Trait** 主要表达一些特性和能力，使用 extends 来选择性的引入某些通用能力。

我们如何设计系统中的对象？这里的对象是 case class。不要使用基础类型（原因很简单，我们的核心是设计类型安全系统，我们希望在程序开始运行之前就能看到和检查出确保这个程序能安全运行，我们希望开始的时候就做到最好）。

函数式编程：对于**纯函数** $y = f(x)$，是没有**副作用**（例如读写数据、人机交互、网络通信）的，即 y 的值仅依赖于 x 任何环境下运行的结果都是一样的而且不会对外部有任何影响。

如果一定有副作用（肯定会遇到的），我们就尝试将副作用推到程序的边缘进行，在程序核心处只做纯函数（至少能对核心内容有更好的掌控，如果中间副作用出了问题对整个 workflow 的影响是比较大的，因为我们要收拾）。下面的 IO Monad 会说明的。

函数式编程和面向对象编程相比，**前者负责处理数据**，后者负责定义对象，显然这两种范式并不是矛盾的，其实是可以垂直互补的，上面就有互补的设计。

Monad：一种“带上下文的容器”，让我们能够把一系列操作给安全的顺畅的链式连接起来，通过 `flatMap`，而过程中不用去处理底层的细节（空值、异常、副作用）。包含**包装**和**链式解包和重组 flatMap**（从盒子中取出值应用一个函数返回一个同类型盒子然后把结果压扁）。

也即上面的 Option[T] 和 Try[T] 其实都是 Monad 概念。下面我们提出 **IO Monad 专门用于解决“副作用”的问题**。IO Monad 把副作用包装成一个“蓝图”/“描述”，我们写一个 `val x = IO(println("Hello"))` 时程序返回了一个 IO 对象（表示计划去 print），只有在最外层显式调用 `x.unsafeRunSync()` 时副作用才会真正发生。也即，代码过程中始终都是纯函数（因为始终在拼接 IO 对象），副作用延迟到程序“边缘”，方便定位和测试。

怎么说，对于习惯了命令式的人来说其实这个 IO Monad 看起来不过如此，就像是把代码整体复制了一遍放在末尾处理然后玩一个小的文字游戏，但是其实，显然 IO Monad 还有很多特殊的功能，这些应用非常厉害，例如 timeout、race 和 resource（以及 IO Monad 最直接的一个好处就是，一个 IO 结束之后最后一定会执行清理逻辑，即无论中间发生了什么 finally 里面最终会释放干净）。

我们要求整个类型安全系统基于 Monad 完成，我们要求我们后端的业务代码最终给一个 IO[Response]（执行计划），我们的后端只负责纯粹的组装，当我们把 HTTP 请求交给后端框架（例如 scala 的 http4s）时，**框架在底层是一个无限循环的事件循环 event loop**，接管了我们的 IO 盒子在合适的时候打开执行里面的副作用然后把真正的 HTTP 响应吐给前端。

对**并发**友好（无锁编程和非阻塞，前者是根本就没有共享的可变状态所以不需要加锁，后者是基于 Monad 的后端框架通常就是异步非阻塞的）。

Cats effect 给纤程 Fiber（150 bytes/fiber，非常轻量级的线程）提供各类原生支持，同时 Cats effect 引入 IO[_] 然后 IO 是一个 Monad。所以推荐使用这个。

我们再次强调选取 Scala 的原因，在 Scala 中函数不是特殊的语法结构，**函数本身是一种普通的类型。**函数式编程的思维方式是，当我们习惯“函数是一个值”之后就可以去掉复杂控制流 if/for，写出“声明式”的优雅设计，“优雅”是函数式编程的核心理念。

---

lec04 微服务概念 + 数据库 10.29

微服务：复杂的后端系统需要**模块化**，即把庞大系统拆分成一个个“微服务”让每个微服务只专注于做“一大类事情”（即单一职责）。

微服务应当包含：**名字**（决定定位）、**数据库**（独自管理的数据，微服务通常强调**数据**私有）、**API 功能**（对外提供的能力）。

微服务的基本原则：**高内聚低耦合**（即微服务内部的功能高度相关紧密联系、微服务之间的依赖关系尽量弱甚至互不干扰）。

API 即应用程序接口，是微服务的“功能说明书”（类似于 agent 的 tool list 一样），由微服务的定位决定用来处理特定数据库数据或者做特定运算。大系统拆成小部门 → 小部门通过窗口 API 办事。

我们要求前后端的数据类型、API 和微服务名字完全一致，这是软件开发中极其重要的**契约 contract** 概念。前后端之间的 API 是严格对应调用的。微服务之间的 API 有上下游数据类型匹配的要求。下面会讲到序列化用于

API 传输数据。

序列化：即把对象转成 json 字符串；**反序列化**，把字符串再转成对象，通常使用 json schema。所有跨网络的 API 调用都需要使用序列化，但我们课程中自己电脑上搭建的系统（都在前端或都在后端时）不用，因为我们实际上是在自己的电脑上跑的同一个进程的，可以直接函数调用（换而言之我们实现的是一个 toy 微服务不是真的微服务）。

Scala 语言中一个流行的序列化/反序列化库 **Circe**，半自动推导，即可以用少量的代码把复杂对象 case class 转成字符串 json。需要的时候就使用。

数据库：基础类型通常本身有对应的类型；复杂类型序列化后写成字符串（可以存成 json 格式，我觉得很合适）。数据库本质是一个存储数据的表格，关心规模性和并发性和高效查询。

关系型数据库（例如 MySQL、PostgreSQL）基础概念：把数据库想象成一个 Excel 表格，然后行代表一条完整的数据/记录，列代表数据的属性/分类。有如下规则：每一列有自己的固定类型且只有基本类型（如果有复杂类型就序列化成字符串 text/json）；每一行有一个**主键 primary key** 相当于每一行数据的身份证号（不允许相同）；**外键 foreign key** 用于把两张表“关联”起来（一个表的列可能是另一个表的主键，通过这种联系可以查出两边的数据）。

---

lec05 前端大杂烩 + 前端架构介绍 10.30

网页时随着时间推移、事件发生而不断变化的动态过程。我们称网页的形式是 $V_t = \mathrm{R}(S_t)$ 其中 S_t 是 t 时刻的状态、R 是渲染函数、V_t 是 t 时刻最终呈现给用户的画面。状态只有发生了**用户事件**（例如点击按钮）或**网络结果**（例如服务器返回新数据）时才会更新（也即，时间流逝本身不改变状态，是**事件**在推动状态的更新）。

JavaScript 负责“交互逻辑”是浏览器执行交互的**底层**语言。TypeScript 在 JS 的基础上有**类型系统**（运行前用类型做基础检查，最终会转成 JS 交给浏览器）。React 负责“构建用户界面”是一个把数据（状态）**组装**成页面（组件）的库。由此可见，**页面是状态的投影**（不直接修改页面 DOM，我们修改的是状态，由 React 自动根据新状态重新渲染页面）；时间只是“事件”的一个维度。

TypeScript 基础语法：`function f(x: number, y: number): number {...}`、`const` 表示变量名绑定的值不可变、`type` 定义一种数据形状、`user.name`、`(x) => x+1` 箭头函数、`[a,b]` 数组结构、`{name: "x"}`、`import / export` **模块化，前者是把其他文件的东西拿进来用，后者是把本文件的东西分享出去**、`<button>...</button>` JSX 语法（React 特有，描述页面上要显示一个按钮，在 TS 文件中通常有后缀 `.tsx`）。

页面数据设计（其实是关于下面 Model 中的数据的设计）：**核心数据**（原状态：必须由外部输入、不能靠计算得到）；**展示型数据**（派生数据，能够根据核心数据算出来）；**常数**（固定不变）。

React 单向数据流，用 MVC（模型-视图-控制器）思想：**事实（Model 核心状态）**有最基础的**数据**；**画面（View 视图）**例如输入框、按钮，**视图是事实的投影**；**操作（Controller 事件处理）**用户触发事件函数**更新核心状态（事实）**。这是一个循环：事件发生 $\rightarrow$ 更新核心状态 $\rightarrow$ 纯计算得出派生数据 $\rightarrow$ 渲染出新视图。

纯函数：给相同输入得到相同输出且执行过程中不产生副作用。组件的“纯净”：组件是一个“装配车间”，接收参数、调用纯函数算好数据、返回 JSX（描述界面的图纸，React 暗自把 JSX 转成真正的网页元素）。后面我们

会说到，副作用（例如发请求、写本地存储）应当放在事件处理函数（例如 onClick）或专门的 Effect（例如 useEffect）中，**组件的渲染过程本身必须是纯净的。**

`const [count, setCount] = useState(0)` 即向 React 申请一个记忆变量 count（初始值 0），下面可以使用 `setCount(c => c+1)` 将对 count 进行修改。

React 的每次渲染是拍快照（**快照机制**），`setCount` 是向 React 提交了一个更新请求（会改变下一次拍快照时的结果），从快照中获取到的信息，始终是在记录快照所在的时刻的信息。后面我们会说到这种相关联的动态信息更新如何操作。**这里其实就体现出 React 必须和函数式编程搭配的合理之处**，如果我们连续两次 `setCount(count+1); setCount(count+1)`，最终的 `count` 值只增加 1，因为第二次使用的 `count` 并没有变化（传递的是固定的值，因为快照机制，对应命令实际等价于把 `count+1` 的值计算出来代入 setCount 里面）；但如果我们连续两次 `setCount(c => c+1)` 传递的就是一个函数（操作指令），React 会把当前最新状态传给它。因此，我们强烈要求 **React 搭配函数式编程来改变数据。**

Hook 钩子（在 React 中）是 React 提供的一系列特殊函数（名字通常以 `use` 开头例如 `useState`，`useEffect`）。Hook 的作用是让普通函数组件拥有“特殊能力”（记住状态、处理副作用、访问 React 的底层机制，普通 JS 函数运行完即重置，给函数配上 Hook 相当于“记忆”和“通讯”）。Hook 只能在代码最顶层，显然不能放在 if 或 for 中（React 判断状态对应的变量是依靠 Hook 调用的顺序，必须足够一目了然）。

上面的 `useState(0)` 就是 **React 给组件添加“记忆”的 Hook。**其中 `setCount` 作为**修改这个值的唯一正确途径**，是 `useState` 的更新函数（不会立刻修改值，而是向 React 发送一个更新请求，这也是我们使用函数式编程的原因，我们可以将一个函数作为请求发送过去）。

`useEffect` 是用来处理“副作用”的 Hook。组件主要职责是“根据数据计算和返回界面 JSX”（纯净操作），这个 Hook 允许我们在组件渲染完成后执行额外操作例如**发送网络请求、操作浏览器 DOM（修改网页标题等）、设置定时器、写 localStorage** 和生成界面无关的外部交互。我们可以控制执行条件（例如某个变量变化时执行、只在组件第一次出现时执行）。

React 的运行逻辑是一个循环（单向数据流，上面其实提到了的）：初始化（第一次渲染）→ 用户交互（触发事件）→ React 收到更新请求（排队和调度，React 把任务放进队列，然后**重新调用对应的组件函数产生下一次渲染**）→ 重新渲染（**产生新快照，这是我们使用函数式编程的原因！**）。如果有 `useEffect` React 会在页面更新后检查依赖项是否有变化，有则执行里面的副作用代码。

在之前 React 的渲染是递归同步执行的，后来 Fiber 出现把渲染工作拆成 Fiber 节点用链表串起来，因此**可以随时暂停、记住当前位置、下次修改。**因此 React 可以在浏览器每一帧 16ms 预算内做一点渲染工作然后将控制权还给浏览器让用户交互优先响应。

`useEffect( () => {document.title="You clicked " + count + " times"; }, [count]);` 第一个参数 `() => {...}` 是要执行的**同步动作**；第二个参数 `[count]` 是依赖列表（含义是只有 `count` 发生变化时才执行，空数组是只执行一次，省略是每次都执行）。**Effect 可以理解成是“组件提交更新后的同步安排”。**以及值得注意的是如果 useEffect 里面由 `return () => ...` 则后面跟着的**清理函数**，会在 React 准备丢弃这个组件（或重新执行这个 Effect 之前）自动调用 return 的函数（例如清理掉开着的计时器）。

（上面是从相对底层的组件开始说起的，下面会开始跨组件的来说）

很多组件和页面会共享同样的数据，**Zustand** 是一个轻量级的状态管理库，创建出来**容器 Store（仓库）**，存在里面的跨越多个共享页面的数据称作**全局状态 Global State**。Zustand 提供了三种接口：`getXXXSnap()` **拍照**（**获取快照**，调用这一瞬间的值，在点击事件里查看当前数据，之后数据是不变的）；`useXXX()` **订阅**（**自动更新**，让组件“盯紧”这个状态，如果有变化 React 会自动触发该组件的重新渲染和画面更新，这适合页面上需要**持续展示的数据**。我们需要在组件函数里调用它）；`setXXX(...)` **更新**（**修改并广播**，修改 Store 里面的值并通知所有订阅了该数据的地方）。

注意这里可能看起来有点奇怪，我们需要声明，`useXXX()` 和 `setXXX(...)` 本身都不是副作用，但是 `setXXX(...)` 会引起副作用。我们的订阅会跟踪并随着变化而更新，但是每次更新都是组件用数据来组装之后渲染，这个过程本身是没有副作用的。当然 `setXXX(...)` 会产生副作用所以我们的 `useEffect` 本身就是因此而生的。

组件展示界面需要用订阅；事件回调（**回调**：平时这个函数不执行，我们点击按钮后系统回过头来调用这个函数）时只需要拍照。注意，由于 React 判断数据是否发生变化的方式是“浅比较”使用数据存储的内存地址，因此当使用 `setXXX(...)` 时（基本类型无关紧要，但**如果是对象，必须使用不可变更新**），需要创建一个新的不可变对象（使用 `const`），然后让旧的对象指向新的不可变对象的内存地址，所以其实 `setXXX(...)` 本身是一种**覆盖**（类似于 LangGraph 的状态一样，但是只允许整体完全更新）。

内存 Store 不是永久的，Zustand 的 Store 默认在浏览器的普通内存里（刷新即重置），如果希望刷新后数据还在需要明确使用浏览器的本地存储 localStorage（可能被用户篡改）和服务器（掌握的数据和权限必须由服务器来核实）。

三种数据传递方式：**Zustand store**（**全局状态，长期持续**）；**组件 Props**（父子传参，适合**父子组件**之间的数据传递，必须从上往下流，如果子件想修改父件的数据就必须调用父组件传下来的 `action` 函数，一个下达传送指令的按钮）；**路由状态 Location State**（带材料跳转，**适合一次性的页面跳转**，只传一次，路由使用 React 的 `useNavigate` 跳转时带上 state，目标页可以使用 `useLocation()` 来取出，但例如用户直接输入网址打开跳转页面可能 state 是空的所以还有 `isCheckoutInput` 来做接收做**校验**）。

（上面说了跨组件数据传递，下面会开始介绍前端的代码结构）

接下来说**前端开发中的代码架构设计和模块化**（相当重要相当重要！！）。我们将按照**“业务模块”**来拆解整个前端；页面拆分的终极目标是：**高内聚、低耦合。**

在过去，前端页面由 HTML（“骨架”：负责整个页面）、CSS（“皮肤”：例如按钮红色蓝色、标题字号、排版格式等）、JS（“肌肉”：点击按钮弹出窗口、从后台拉去数据，决定“页面能干什么”的逻辑）拼起来。TS 其实只是 JS 增加了类型检查，我们的 TS 最终会被编译成 JS。而 TSX = TypeScript + XML（即 HTML 标签）将 HTML 骨架和 JS 逻辑融合在一个文件里（例如动态生成界面，“如果用户登录了就显示头像”的逻辑 + HTML 标签）。

其实这里真的得说的细致一点因为我们最好还是要对前端代码做足够的 code review。首先传统的 HTML 是副作用的重灾区因为我们想让界面动起来就必须操作 DOM。而 React 的 TSX 则是我们写好**“声明”**，React 在底层帮我们完成了“修改 DOM”这个副作用，也即 **React 在本质上是帮我们管理副作用的**，我们负责写纯函数（根据数据描述 UI）React 负责安全执行。

在 React 中，网页是像“俄罗斯套娃”一样拼起来的，例如我们结构图中的 `index.tsx`（总装图）：

```jsx
// 假设这个是父组件，即一个总装图
function BookListPage() {
  return (
    <div className="page-container">
      <h1>图书馆</h1>

      {/* 把过滤条拼进来 */}
      <BookFilters />

      {/* 把表格拼进来 */}
      <BookTable />

      {/* 把分页条拼进来 */}
      <PaginationBar />
    </div>
  );
}
```

我写了 `<BookTable />` 这个东西之后 React 就会去执行 BookTable.tsx 这个函数，BookTable.tsx 运行后返回一堆 `<table>,<tr>,<td>` 标签，React 会把这些返回的标签“嵌套”到父组件的 `<div>` 里，层层嵌套拼成一棵完整的 DOM 树（一个很大的 HTML）。

于是有一个非常好的结构（“页面级”拆分结构）：

```
pages/library/BookListPage/
├── index.tsx            // 总装图：负责组装整体，不写具体细节
├── components/          // 局部组件
│   ├── BookFilters.tsx  // 筛选条
│   ├── BookTable.tsx    // 表格
│   └── PaginationBar.tsx// 分页
├── functions/           // 纯函数：只负责计算（如过滤、分页计算），不发请求
│   └── filterBooks.ts
└── hooks/               // 自定义 Hook：负责状态管理、接入外部系统
    └── useBookList.ts
```

我觉得至少得看懂上面前面的 `index.tsx` 的组装才能看懂下面的这个结构的示意。其中 `index.tsx` 是总装图（只负责把各个零件拼起来，不干具体的活）、`functions/` 里面全是**纯函数**、`hooks/` 负责管理状态、处理副作用。

拆分了单个页面，还需要有对整个项目的目录组织，通常来说我们的文档最好统一采用同样的标准（有个专有名词形容叫做“分形结构”），即我们局部和整体的项目布局都是一样的，都是按照【组件、逻辑、样式】在每一层都保持一致。

我们用 `components/ui` 放纯 UI 基础组件（按钮、输入框）、用 `infrastructure` 放基础设施（例如网络请求、路由配置拦截等）、用 `shared` 放从纯粹的工具函数 Utils 和全局的常量、通用的类型定义 Types。而页面级 Pages 就只关注对应页面自己的事情。

注意，我们在公共目录（components, infrastructure, shared）增加代码时，必须找到真实的至少两个真正使用者。我们必须避免“过度抽象”，可以多写一些重复代码但绝对不要制造虚假的公共耦合（等真正出现重复使用时再抽离是可行的）。

（上面介绍了前端的项目架构，下面我们开始说到 CSS）

CSS 的本质是“排版”，全程是 Cascading Style Sheets 层叠样式表。CSS 关心选择器 Selector 决定作用对象、属性 Property 选中对应属性、值 Value 将属性的值进行修改。“层叠”的含义是多条规则作用在同一个元素时，浏览器会按照**来源、优先级（权重大小）、顺序**来决定。

CSS 中我们可以给特定的按钮加上一个“类名” `<button className="primary-button">` 给 button 赋予专属名字。这样我可以 `.primary-button { background: blue; }` 来指定这个按钮进行修改。如果要修改所有 button 元素可以用 `button { background: red; }`。

交互状态例如 `:hover` 称作**伪类选择器**，代表用户的某种**特定交互状态**，例如 `.primary-button:hover { ... }` 意思是当用户把鼠标**悬停**（停留在）带有这个类名的按钮上时请应用大括号里的样式。

盒子模型 Box Model：网页排版的基石，浏览器会把所有 HTML 元素（文字图片按钮）都看作是一个个矩形的“盒子”，一个盒子从内向外有内容 content、内边距 padding、边框 border、外边距 margin。稍微注意一下 CSS 的盒子模型里，我们通常推荐使用 `box-sizing: border-box` 这样设置 `width: 300px` 就包含了“内容+内边距+边框”的最终宽度（如果按照默认的 `box-sizing: content-box` 设置的 `width` 就仅仅是里面内容 content 的宽度）。

也即，CSS 通过**规则**精准选中元素，用**盒子模型**（内容、内边距、边框、外边距）来排版。

CSS 中有两个用于页面布局的最核心工具，**Flexbox 弹性盒子**（用于处理一维的排队 `display: flex` 开启）和 **Grid 网格布局**（用于处理二维网格，`display:grid` 开启）。

Tailwind CSS 用于解决一个传统 CSS 容易遇到的问题：我们总是需要给元素起类名然后在另一个 CSS 里面写 `.类名` 来回切换。而 Tainwind 提供了海量的“原子类 Utility Class”，我们可以直接用这些预设的小工具类瓶装样式，例如 `<div className='p-4 bg-white rounded-xl shadow-md">`（我觉得其实很方便，虽然可能会让 HTML 和 JSX 的代码变长）。

组件库（例如 shadcn/ui 库）提供了按钮弹窗表格的一些基础外观和样式，我们可以直接使用。当然组件库只负责“外观”不任何负责“业务规则”（这是我们 TS 要干的事情）。

Vite：极速开发体验，传统写代码时修改一点东西浏览器都要缓慢刷新出效果，Vite 让我们在 IDE 里改好代码之后能瞬间把改动的那一小块推送到浏览器。在开发环境中 Vite 可以飞速预览，在生产构建（上线给用户看）中 Vite 会把代码压缩打包成浏览器可以高效运行的文件，交给服务器托管。

但是需要注意，Vite 是不做类型检查的，对于我们的 TypeScript 代码 Vite 会暴力拆解去掉所有类型，也就是说“飞驰时不管但在刹车交货前严格检查”。当然可以通过在 `package.json` 里面配一条命令 `"scripts":{`

`"check-types": "tsc --noEmit" }` 来让另一个编译器 tsc 来检查语法和类型（不生成任何新文件）；也可以在 Vite 配置中集成插件例如 `vite-plugin-checker`。

也即，用 Tailwind 写样式；调用现成的 UI 库用组件；使用 Vite 跑代码和补充静态类型检查。

（真离谱啊在九十分钟里讲前端这么多概念是不是还是对前置知识基础的要求有点高啊）

---

lec06 前后端架构介绍 10.30

其实 lec06 是 code review，我们根据 sample 做一下后端架构的介绍（具体的 scala 基本语法在前面介绍的比较足够）和前后端架构配合。

后端的示例的目录结构：

```
Main.scala 程序入口（负责启动整个 HTTP 服务器）
infrastructure/ 基础设施层（存放数据库连接、服务器配置等，所有业务都能用但不包含具体业务逻辑）
shared/ 工具共享层（存放经过验证的通用小工具）
services/（核心）：业务服务层。最最核心，我们按业务继续拆分文件夹。
```

一个业务模块内部的结构，我们按照四个维度拆分：

- **objects 业务对象**：定义数据的形态和基础业务规则。
- **api 应用接口**：对外提供“服务菜单”。
- **routes 路由**：负责把外部发来的 HTTP 请求映射搭配 api 的具体操作上。
- **tables 数据表操作**：专门负责和数据库打交道（增删改查）。

有多个业务模块，我们用一个**总路由 ApiRouter** 即 `allRoutes = combineRoutes(...)` 组合起来，总路由负责把唯一的网站入口的 HTTP 请求路由到子模块里面。

模块之间相互的调用，通过公开的 API 或 Table 方法调用（即使两个模块在同一个程序里我们也只使用 API 而不要亲自动手去查数据库）。

如果未来系统升级我们需要把读者模块拆成独立的服务器，我们原本的内部函数调用就需要改成发送网络请求，但只要“调用边界”足够规范（例如接口定义清晰）迁移成本就会很低。

（上面是整体的目录结构，下面开始专注类型安全的代码微观层面）

我们写 `function createBorrow(readerId: String, copyId: String)` 时如果写反了两者编译器是不会报错的，但是我们希望**类型安全系统**能够指出这个问题。我们可以给基础类型套上一层“马甲”来区分，用 `case class`：`case class ReaderId(value: String)` 和 `case class CopyId(value: String)` 这样函数必须接收 `ReaderId` 和 `CopyId`。

业务对象可以使用抽象类 abstract class 和 trait 来分类和赋予对应的能力，这个确实还是和 Grothendieck 有点关系的。我们区分**乘积类型 Product Type**（同时有的 AND）和**和类型 Sum Type**（几种之一的 OR）。只有为

了区分“重要业务边界”时才上类型定义（避免“过度抽象”），按照业务语义分组（例如 Policy、Content 等）。

数据库的并发的解决方案：事务 Transaction（全部生效或全部作废即回滚）；“原子操作”（将检查和操作合二为一在数据库层面强行约束，或者上锁）。

前后端对齐：前端（浏览器端）和后端（服务器端）需要约定好“对话规则”，网络传输只“传递字符串”，**必须用“契约 API Contract”来约束双方，在边界处做好严格数据转换。**也即我们需要保证前端调用 API 的能力和双方交换的数据对象在业务含义上完全一致。

前端中在 api 里有负责“动作对接”写如何向后端发送请求（用的 HTTP 和 URL 是什么）；objects 有负责“数据对接”定义后端传回来的数据形态；infrastructure/httpClient.ts 负责“底层通信”，统一处理添加身份信息 Token、读取响应、处理网络错误。

DTO Data Transfer Object 是数据传输对象，为了网络传输需要临时定义一个“快递盒子”。序列化和反序列化。注意 ID 需要是字符串（否则会丢失前导 0），事件必须包含时区信息的 ISO 8601 标准格式，前端拿到后必须转换成当地时区。我们可以在 TypeScript 中声明 `type BorrowSuccessDto = { ... }` 但是 TypeScript 在浏览器运行时不会检查类型（所以拿到网络返回后一定要进行运行时检查）。

字段名和含义？类型和单位？缺失和空值？集合？枚举和分支？请求入口？正常和失败？版本变化？**契约还要定单位的精度和空值处理。**

浏览器地址栏的 URL 本质上都是字符串，**架构原则是在入口处转换，业务内部只用强类型对象。**即在 AppRouters 里把 URL 里的字符串解析成带类型。也即，把字符串校验这种脏操作隔离在业务逻辑外面，业务代码只处理干净的业务对象。

前端应当做好状态机设计，即在用户点击按钮之前前端就应当想好可能的状态，例如：

```
type BorrowUiState =
  | { kind: "idle" }                                  // 空闲，没点
  | { kind: "submitting" }                            // 提交中，按钮转圈
  | { kind: "succeeded"; loan: BorrowSuccessDto }     // 成功，拿到了借阅凭证
  | { kind: "failed"; message: string };              // 失败，拿到了报错信息
```

这是一种**可辨识联合**，杜绝各种中间状态，其实就和前面 scala 的 enum 是一个性质（这是**类型收窄**，避免介于中间灰色地带的中间态，比如成功但拿到报错信息的情况）。

前端到数据库：React 组件层（用户交互）→ 前端基础设施（网络发送，按照契约包装网络请求并附加身份信息）→ 后端路由层（门卫安检，解析请求识别 Token 转成安全对象，负责过滤脏数据）→ 后端业务计划曾（调用 api）→ 后端相应层（把数据库里的复杂对象打包成 DTO，如果业务失败要打包错误信息返回）→ React（前端拆开，修改状态）。

时间差：后端保证**“幂等性”**并让同一个请求发送多次保持和发送一次效果一样。

前后端架构的文档应当把**业务规则、边界、动作、反馈**都说清楚（例如权限、数据、前端等）。由此可以看出前后端架构设计是一件多么危险的事情，和类型安全系统在其中蕴含的巨大潜力和发挥的巨大作用。